Toward End-to-End Analog Design Automation with ML and Data-Driven Approaches

(Invited Paper)

Supriyo Maji[1], Ahmet F. Budak[1,2], Souradip Poddar[1], David Z. Pan[1]
[1]*ECE Department, The University of Texas at Austin, Austin, TX, USA*
[2]*Analog Devices, USA*
{supriyo.maji@austin, ahmetfarukbudak@, souradippddr1, dpan@ece}.utexas.edu

2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC) | 979-8-3503-9354-5/24/$31.00 ©2024 IEEE | DOI: 10.1109/ASP-DAC58780.2024.10473840

***Abstract*— Designing analog circuits poses significant challenges due to their knowledge-intensive nature and the diverse range of requirements. There has been limited success in achieving a fully automated framework for designing analog circuits. However, the advent of advanced machine learning algorithms is invigorating design automation efforts by enabling tools to replicate the techniques employed by experienced designers. In this paper, we aim to provide an overview of the recent progress in ML-driven analog circuit sizing and layout automation tool developments. In advanced technology nodes, layout effects must be considered during circuit sizing to avoid costly rerun of the flow. We will discuss the latest research in layout-aware sizing. In the end-to-end analog design automation flow, topology selection plays an important role, as the final performance depends on the choice of topology. We will discuss recent developments in ML-driven topology selection before delving into our vision of an end-to-end data-driven framework that leverages ML techniques to facilitate the selection of optimal topology from a library of topologies.**

I. INTRODUCTION

There are mainly three steps in analog circuit design process: topology selection, sizing, and layout [1]. Each step can be time consuming and may suffer from non-optimality. Since the early '90s, significant efforts have been directed towards automating analog circuit design process. While some progress has been made in automating part of the flow and in easing efforts to design specific classes of circuits, the goal of a fully automated flow that works across a diverse range of circuits remains unachieved.

Traditional approaches to circuit sizing can be categorized into equation-based and simulation-based methods. Equation-based method uses analytical equation to model circuit performances which are then solved by optimization toolbox to meet some target specification [2]. It is computationally efficient, but the optimality of the solution is contingent upon the nature and accuracy of the models. For example, a globally optimal solution can be found by solving posynomial equations of circuit performance using geometric programming [3], but such solution may not be accurate in deep-submicron technology [4]. Simulation-based method, on the other hand, integrates circuit simulator within an optimization engine to search the design space [2]. Since circuit simulation is expensive, especially for dc and transient simulation, this method suffers from scalability issue.

Bayesian Optimization (BO) serves as a framework for the model-based optimization of noisy and costly black-box function [5][2]. BO is particularly useful for problems that do not have closed-form expression of the objective function, but have sampled data points to guide the optimization process. It employs two important components to navigate the solution space efficiently, a surrogate model that works as a proxy representation of the costly simulation process by taking the whole history of optimization into consideration, and the acquisition function that works as a cheap-to-evaluate utility function to guide the sampling decision. The Bayesian optimization framework has shown potential in approximating the global optimum with a relatively small number of evaluations. Gaussian process regression (GPR) is the most commonly used surrogate model due to its greater learning ability compared to other alternatives, e.g. artificial neural networks, radial basis functions [6]. However, time complexity of GPR modeling is $O(N^3)$ with respect to the number of training datapoints N, which can hinder the scalability of BO [2][7].

In recent times, reinforcement learning (RL) has emerged as an effective approach for circuit sizing. AutoCkt [8][9] is a sparse sub-sampling RL technique aimed at optimizing circuit performance through discrete actions within the solution space. This method trains an RL agent to acquire knowledge of the entire design space, enabling it to make informed decisions during the design process. However, AutoCkt needs a substantial training dataset, often involving large number of (up to 10^5) simulation samples [2][7]. DNN-Opt [10] is a deep neural network-based black-box optimization algorithm. DNN-Opt draws inspiration from the structure of continuous action space actor-critic algorithms commonly found in the RL algorithms. This structure is modified to handle the constrained circuit sizing problem by merging the strengths of reinforcement learning and bayesian optimization tech-

niques. It also employs a population-based convergence scheme to improve sample efficiency. DNN-Opt has a feedforward neural network which is trained to approximate circuit performance. This trained network allows prediction of unseen design points in inference mode improving the efficiency of the search mechanism. APOSTLE [11] is a faster version of DNN-Opt, leveraging parallel computing resources to expedite the search process. Unlike conventional circuit optimization approaches, GCN-RL [12] has the ability to transfer knowledge across technology nodes. Since the design principles remain consistent across different technology nodes for the same circuit topology, an RL agent trained on one technology can optimize the circuit in another technology.

The automation of the layout generation process for analog circuits has been an active area of research for many years. Nonetheless, the adoption of automated layout tools within industrial flow has been limited. This is primarily due to the fact that analog layout strategies lack a standardized design methodology followed in digital design. Analog circuits often demand the expertise of an experienced layout designer to manually layout the circuit to satisfy various requirements including, symmetric placement and routing, coupling, IR drop, electromigration etc. In recent times, ML-driven tools, such as MAGICAL [13][14] and ALIGN [15][16], have emerged as solutions capable of achieving layout quality on par with that of experienced designers.

In traditional circuit sizing approaches, effects of layout are not generally considered. However, in advanced process nodes, the performance of analog circuits heavily depends on layout implementation. Factors such as layout-induced parasitics and mismatch effect can substantially impact the final design performance. Consequently, factoring in layout effects when optimizing design parameters would enhance the robustness of the sizing result. In recent years, progress has been made in layout-aware sizing using ML algorithms [17][18][19][7][20].

In a broader context of analog design automation, there may be an additional step involving topology generation, which is the creation of new circuits. The steps of topology generation, selection, and sizing are sometimes together called topology synthesis [21][22]. There have been efforts to compile libraries of some analog circuit topologies, such as Op-Amps [21][23], data-converters [24], and power converters [25]. For instance, MOJITO [21] and CktGNN [23] are libraries comprising over $100K$ and $10K$ Op-Amp topologies, respectively.

Within the end-to-end analog design automation flow, the selection of the circuit topology is an important initial step, as the quality of the final result depends on the choice of topology. Although there have been significant efforts over the last few decades to automate the topology selection process, their practical implementation has been limited by long runtime due to in-the-loop simulations [21][26], or the demand for extensive design expertise in equation-based methods [22][27][28][29]. Recently, ML algorithms have been employed to simultaneously handle topology generation, selection and sizing [23][30].

Our vision centers on a novel design automation flow that decouples the task of topology selection and topology sizing. Given a library of topologies, we see an opportunity to leverage the cutting-edge circuit sizing and layout automation tools to facilitate the creation of an expansive pre-sized topology library, comprising a diverse array of topology design data points. This forms the basis of our data-driven approach. The objective is to find the most optimal design point from this rich library given only user specifications. Given the power of ML algorithms in extracting valuable insights from the vast dataset, we discuss ML algorithms that can facilitate the selection of the optimal topology from the pre-sized library.

The rest of the paper is organized as follows. Section II and III discuss recent progress in ML-based analog circuit sizing and layout automation tools development, respectively. In Section IV, we delve into ML-based works related to topology selection. In the same Section, we discuss our vision for a data-driven, end-to-end design automation framework. The conclusions are drawn in Section V.

II. ML Approaches to Circuit Sizing

Learning-based tools leverage machine learning algorithms to solve analog design problems. Among various ML algorithms, reinforcement learning (RL) stands out for its higher training efficiency and better modeling capacity, therefore can solve complex decision-making problems across various systems. In the RL framework, an agent interacts with an environment, receiving rewards based on its actions in each state. The ultimate objective is to maximize the cumulative reward. With a large amount of data and enough exploration, RL demonstrates superior performance. We discuss a few key works in this field.

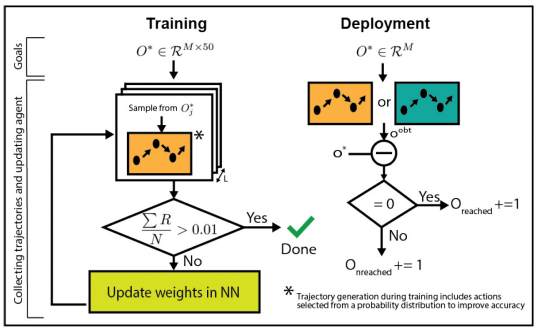


Fig. 1: Training and deployment process in AutoCkt [8][9].

A. *AutoCkt* [8][9]

AutoCkt trains an RL agent to have the knowledge of the entire design space. It helps the agent to choose the next best action at any point during optimization. Suppose there are N sizing parameters to be tuned to optimize M design specifications. Therefore, parameter space is $x \in Z^N$ and the design specification space is $y \in R^M$.

State The state vector contains the target specification, current performance, and current parameter. The reward of each trajectory is the accumulation of reward in each step, which is,

$$R = \begin{cases} r, & r < -0.01 \\ 10 + r, & r \geq 0.01 \end{cases} \quad (1)$$

Where,

$$r = \sum_{i=1}^{M-T} min\{\frac{o_{pt_i} - o^*_{pt_i}}{o_{pt_i} + o^*_{pt_i}}, 0\} - \sum_{j=1}^{T} \epsilon\{\frac{o_{th_j} - o^*_{th_j}}{o_{th_j} + o^*_{th_j}}\} \quad (2)$$

Where, o_{pt} represents hard constraint design specifications and o_{th} represents design specifications that are being minimized. The reward is a measure of how close the current circuit performance is to the target.

Action There are three discrete actions in the set of available actions: increment, decrement, or retain the same parameter values.

Agent The RL agent is implemented by 3 layers, each having 50 neurons.

The agent learns to design a circuit through many trajectories, with each trajectory either reaching the target specifications or terminating upon reaching the maximum number of steps. After the training phase, the agent is deployed to navigate unfamiliar design space. The effectiveness of its training significantly influences the agent's success rate during deployment. Prior to training, the target specifications are randomly selected from the expansive design specification space. Post-training, a distinct set of randomly generated target specifications is employed to assess the agent's competence. The degree of success achieved in realizing these arbitrary target specifications serves as a metric for evaluating the agent's circuit design capabilities. The training and deployment process in AutoCkt is illustrated in Fig. 1.

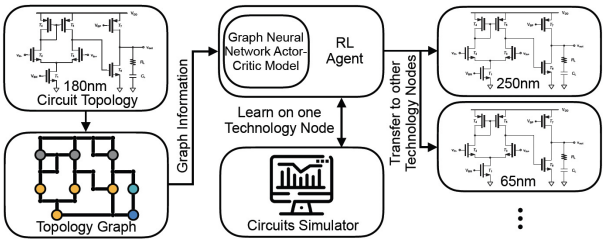


Fig. 2: GCN-RL-based transferable circuit sizing [12].

B. *GCN-RL* [12]

In contrast to conventional optimization approaches, the method presented in [12] has the ability to transfer knowledge across technology nodes, see Fig. 2. This is based on the fact that fundamental design principles do not change across technologies when considering the same circuit topology. GCN-RL maps the circuit to a graph, where transistors are represented as nodes and wires as edges. Connection relationships between components in the circuit are learned with Graph Convolutional Network (GCN), which is used by the RL agent during optimization. In the RL framework, circuit simulator serves as the environment and a Figure of Merit (FoM) serves as the reward signal. The RL agent iteratively refines circuit parameters to optimize the FoM.

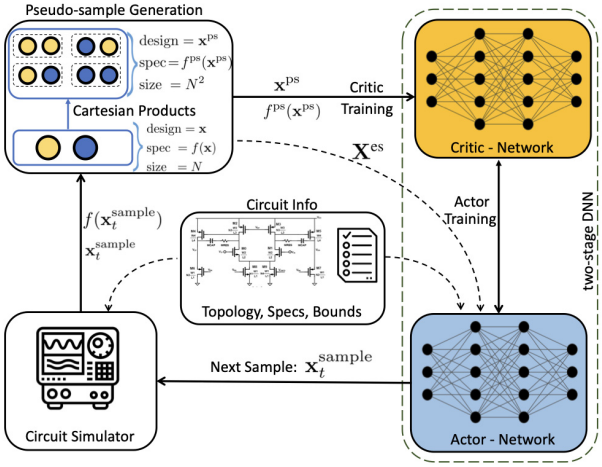


Fig. 3: Core architecture of DNN-Opt [10].

C. *DNN-Opt* [10]

DNN-Opt is a deep neural network-based black-box optimization algorithm that draws inspiration from the structure of reinforcement learning. To handle the circuit sizing problem, it adopts and modifies the neural network framework found in continuous action space actor-critic algorithms in the RL community. In DNN-Opt, the approximation of neural network performance metrics, as originally introduced in [6], is utilized to serve as the critic-network. This critic-network undergoes training through data generated via a pseudo-sampling mechanism. Subsequently, actions are determined by the actor-network, which dictates the next sample point in the optimization process. In this context, the state representation of the environment corresponds to the vector of design variables, while the actions represent change vectors for these design variables. To gain a deeper understanding of how DNN-Opt works, let us delve into the training and utilization of critic- and actor-networks.

Critic-Network Since critic-network is the regression network for circuit performance metrics, it is trained

by the mean squared error expressions obtained by the difference between network predictions and real simulation outputs. For a data batch of size N_b, the loss expression for critic-network, Q, parameterized by θ^Q is given as follows:

$$L(\theta^Q) = \frac{1}{N_b(m+1)} \sum_{k=1}^{N_b} \sum_{l=1}^{m+1} (Q(x_k, \Delta x_k)^l - f(x_k, \Delta x_k)^l)^2 \quad (3)$$

where, $Q(x_k, \Delta x_k)^l$ is the critic-network's approximation for k^{th} sample's l^{th} performance and $f(x_k, \Delta x_k)^l$ is the SPICE simulated value for the same design performance pair.

Actor-Network Once the critic training is done, actor is trained based on the fixed critic. The analogy here is that critic-network behaves as a surrogate model for the design space and actor-network conducts a search within this model to find potential good designs. The training of actor-network has two main components stated below.

$$L(\theta^\mu) = \frac{1}{N_b} \sum_{k=1}^{N_b} (g[Q(x_k, \mu(x_k|\theta^\mu))] + ||\lambda * viol_k||_2) \quad (4)$$

where, $\mu(x_k|\theta^\mu)$ is the proposed parameter change vector Δx_k by the actor-network. The second term involves penalizing actor proposals to restrict the search within a defined region, the boundaries of which are determined by the population of 'elite' solutions. This regularization forces the population to converge to a reasonable solution. In this way, DNN-Opt mimics the convergence behavior of population-based methods (such as evolutionary). Once the next sample is determined by actor-network, it is sent to the circuit simulator for evaluation. The population is updated with the new evaluation result, and the next iteration starts. The optimization continues until either target specification is met or the simulation budget is consumed. The core architecture of DNN-Opt is shown in Fig. 3.

D. Challenges and Opportunities in Circuit Sizing

The simulation-based circuit sizing approaches that leverage machine learning techniques require a substantial amount of simulation data to train neural network models, often on the fly [10][7]. These models approximate the relationships between design variables and performance metrics [10]. However, simulating large and complex circuits can become computationally expensive. Maji et al. [31] introduce the concept of multi-level abstraction for modeling analog circuit performance metrics. At the higher level of abstraction, referred to as macro model, they model circuit performance as functions of device parameters such as transconductance (g_m), drain conductance (g_d), and overdrive voltage (V_{ov}) and at the lower level of abstraction, they model device parameters as functions of device sizes and biases.

This multi-level abstraction can be used in ML-assisted optimization framework to reduce the runtime cost of training. To accomplish this, models of small-signal parameters can be developed using polynomials, or by leveraging high expressive power of neural networks [32]. This is a costly process but is a one-time effort, provided the technology remains constant. For performance modeling on the fly, small-signal representation of the circuit can be used as it is faster than SPICE simulation. This can effectively bypass the need for SPICE simulation in generating training data. Although the SPICE simulation may still be necessary for the accurate evaluation, the cost of such evaluations is relatively less when compared to the training process.

III. ML Approaches to Layout Automation

In recent years, significant efforts have been devoted to automate analog layout generation process [33], leading to state of the art tools ALIGN [15][16] and MAGICAL [13][14]. We discuss some of the key ideas within these tools.

A. ALIGN and MAGICAL

ALIGN and MAGICAL share a common approach, encompassing three main stages in their flow: (1) layout constraint generation such as symmetry, (2) circuit placement, and (3) circuit routing. These tools employ ML algorithms to reduce human effort significantly while generating layout of quality comparable to those created by humans.

1) Layout Constraint Generation: Analog layout synthesis often necessitates matching and placing certain circuit elements symmetrically. However, the symmetry requirements can vary widely between circuits, making them challenging to address using traditional algorithms. ALIGN employs a hierarchical approach through graph neural networks (GNNs) to efficiently extract multiple axes of symmetry within a circuit. It utilizes an undirected bipartite graph to represent the circuit netlist. An important feature of the algorithm is its ability to identify arrays of recurring structures. In some circuits, these recurring structures may not be exact duplicates and can only be identified through approximate graph matching. ALIGN's approximate matching algorithm is based on graph edit distance, which gives a measure of similarity between two graphs.

MAGICAL leverages unsupervised inductive graph-learning-based methodology for both system-level and device-level symmetry constraint extraction, making it applicable to general analog design. The process starts with the transformation of the input circuit netlist into a heterogeneous multigraph. Within this multigraph, each vertex is assigned a feature vector that considers device types and their associated parameters. After this initial pre-processing, actual constraint generation occurs over three steps:

Unsupervised Inductive GNN Learning This step samples and aggregates neighboring features for each vertex, extracting precise structural information.

Circuit Feature Embedding This step determines the features of each subcircuit by identifying representative sub-structures within the circuit graph.

Classification This step computes the cosine similarity between potential symmetry pair candidates to generate the symmetry constraints.

2) Placement: ALIGN and MAGICAL both employ a combination of non-linear/linear programming techniques to optimize wire length, area, and overlaps during the placement of circuit blocks. These optimizations are performed iteratively, taking into account the critical layout constraints, such as symmetry. Given the complex nature of analog circuit behaviors, the extraction of performance-relevant features from the placement process becomes important. Placement performance can be impacted by both the specific placement location and the overall circuit topology. For instance, the mismatch of differential input pairs can have a more pronounced effect on offset than the load. ALIGN integrates compact ML-based models as a component of the cost function, enabling fast prediction of the quality of a candidate layout. This predictive capability is used to reward or penalize the optimization cost function based on whether the layout meets or fails to meet the performance specifications. MAGICAL employs a 3D-CNN model to predict the placement quality of circuit layouts, leveraging well-crafted placement features. To enhance data efficiency for practical applications, a transfer learning scheme is introduced, significantly reducing the required amount of data.

Inserting wells is an important task within the analog layout process, as the geometries of the wells have a significant impact on circuit performance through phenomena like the well proximity effect (WPE), a type of layout-dependent effects. MAGICAL's well generation framework, known as WellGAN, is guided by a generative adversarial network (GAN) model. In the training phase, WellGAN leverages an existing layout database to construct a conditional GAN model, which is subsequently employed during the inference stage. The trained GAN model effectively predicts the well region and guides the well generation.

3) Routing: ALIGN employs a constructive router that uses an integer linear programming formulation and the A* algorithm, this works particularly well for sparse designs. For congested design, ALIGN uses satisfiability-based detailed router. MAGICAL adopts the A* search algorithm during detailed routing to find perfect symmetry whenever possible, or cross symmetry and partial symmetry otherwise, to maximize the degree of symmetry of symmetric net pairs. To enhance performance, MAGICAL uses a variational autoencoder (VAE) based machine models to learn the routing strategies from manual layout database and apply the learned knowledge to guide the automatic router, mimicking the sophisticated manual layout of experienced designers.

B. Layout Aware Sizing

In a traditional analog design automation flow, circuit sizing is followed by automatic layout generation. After including the parasitics and other layout dependent effects, if the target specification is not met, the process is repeated with additional margins added to address the violating specification. Factoring in layout effects when sizing the devices not only enhances the robustness of the sizing result but also minimizes the need for costly rerun of the flow. We explore recent research that uses ML techniques for layout aware sizing.

BagNet [17] is a DNN-aided, layout-aware, evolutionary algorithm for circuit optimization. It contains three main components: (1) an evolutionary core engine to generate offsprings; (2) a layout generation tool, the Berkeley Analog Generator (BAG) [34]; and (3) a DNN model acting as an oracle to compare two designs. The layout generation tool enables the framework to consider parasitics from layouts, and the oracle allows the framework to be more efficient. The efficiency boosting from DNN is especially important as the layout generation and post-layout simulation is costlier than schematic simulation.

Recent advances in machine learning algorithms have paved the way for accurate estimation of layout parasitics directly from circuit schematic through data-driven and statistical methods. ParaGraph [18] presents a technique utilizing GNN model for the prediction of net parasitic capacitance by transforming circuit schematic into heterogeneous graph. MLParest [19] leverages random forest models trained on extracted features from circuit schematic to predict both effective resistance and lumped parasitic capacitance, contributing to enhanced accuracy in prediction. Both methodologies have successfully demonstrated a reduction in errors when comparing pre-layout and post-layout simulation results.

The layout aware sizing framework proposed by Budak et al. [7] takes a different approach to parasitic prediction compared to [18][19][20]. While [18][19][20] employ less accurate parasitic predictors, necessitating further validation of final post-layout results, [7] runs post-layout simulation based on the actual layout generated by MAGICAL. BagNet, and [7] both integrate the layout generator within the circuit sizing tool, however, unlike BagNet, [7] exploits correlation between schematic-level simulations and post-layout simulations during the optimization of design variables to improve efficiency. [7] does not assume the preexistence of any dataset, all training data are generated on the fly during the optimization. To model design

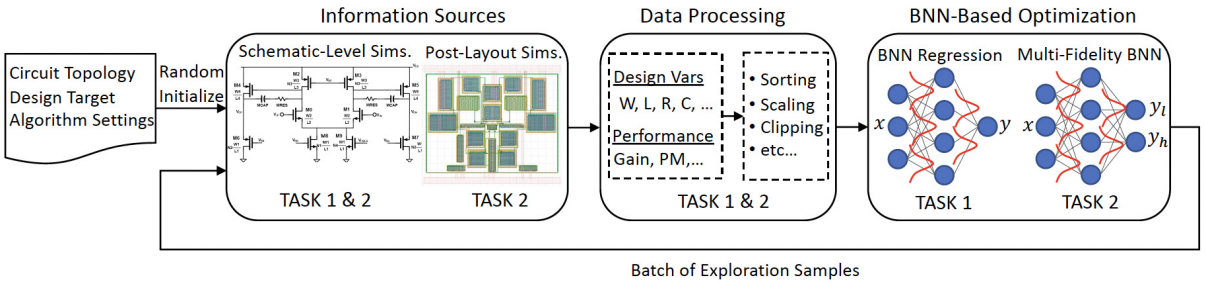


Fig. 4: Layout aware circuit sizing framework proposed in [7].

performances, Bayesian Neural Networks (BNNs) are employed. The choice of BNNs is motivated by their capacity for error quantification, as compared to Deep Neural Networks. BNNs have proven to be effective in managing smaller datasets and in mitigating overfitting [35]. This characteristic is important for enhancing scalability and overall performance. [7] also proposes a batch-optimization framework and a design space sampling strategy compatible with BNN modeling and employs a co-learning scheme to reduce the need for resource-intensive post-layout simulations, improving overall efficiency. For an illustration of the BNN-based layout-aware sizing framework, refer to Fig. 4.

C. Challenges and Opportunities in Layout Automation

In a typical layout-aware sizing environment [7], every time the sizing algorithm refines the device sizes, the entire layout is generated from the ground up, including cases where only few devices are resized. This practice underscores the need for developing a layout engine that allows incremental changes to floorplan/placement and route, affecting only a small portion of the chip's layout while leaving the rest undisturbed. In many instances, resizing the devices may not be necessary. For example, addressing coupling noise affecting sensitive devices may just need identifying the specific problematic net and the sensitive device and providing extra shielding or adjusted net spacing. We believe that along with the incremental layout capability, developing techniques capable of pinpointing the precise area or component within the layout causing certain circuit performance to degrade is crucial in minimizing costly reruns of the flow and in improving the overall quality of the design.

IV. ML Approaches to Topology Selection

In recent times, ML algorithms have been used to simultaneously handle topology generation, selection and sizing within a unified framework. We discuss some key ideas of CktGNN framework presented in [23], and AnGel framework presented in [30] next.

A. CktGNN

CktGNN is a Graph Neural Network based approach that can simultaneously handle topology generation, selection and sizing. By recognizing that the graph structure is a natural representation of circuit, CktGNN encodes the circuit using a two-level GNN framework, where circuits are represented as combinations of non-overlapping subgraphs in a known subgraph basis, which helps to reduce the topology search space. In the transformed graph $G(V, E)$, each node $v \in V$ represents a subgraph g_v from the input graph (circuit). CktGNN leverages inner GNN, consisting multiple undirected message passing layers followed by a graph pooling layer, to learn subgraph representation h_v. Inner GNN captures the contextualized information within each subgraph, thereby enhancing the representation ability of CktGNN. Undirected message passing also provides better parallelizability than directed message passing, resulting in enhanced encoding efficiency. CktGNN leverages outer GNN to perform directed message passing, where the aggregate function $\mathcal{A}$ and the update function $\mathcal{U}$ use a gated summation and a gated recurrent unit (GRU), respectively.

$$a_v = \mathcal{A}(\{z_u | u \in \mathcal{N}(v)\}) = \sum_{u \in \mathcal{N}(v)} g(z_u) \otimes m(z_u) \quad (5)$$

$$z_v = \mathcal{U}(concat(x_v, h_v), a_v) \quad (6)$$

Here, z_v is the hidden representation of node v, m is a mapping network, and g is a gating network. In the GRU, x_v is the one-hot encoding of the subgraph type. The outer level GNN processes nodes in G in a topological order and uses the hidden representation of the output node as the graph representation. CktGNN uses circuit simulator to evaluate the performance of generated circuit topology in real-time. To evaluate the effectiveness of the proposed network, an Open Circuit Benchmark containing $10K$ Op-Amps is introduced.

B. AnGeL

AnGeL is a fully-automated analog circuit generator that leverages neural network models for topology generation, selection and sizing. AnGeL employs a hierarchical approach to determine the specifications of sub-circuits, based on the specification of the parent circuit. This hierarchical approach enables AnGeL to design circuits with many parameters fast while ensuring high accuracy. AnGeL utilizes two key neural network models for topology generation/selection:

A classifier to select the most suitable high-level circuit topology from an array of candidate options.

A regressor to break down the overall specifications into sub-circuits specifications for each high-level topology.

For topology sizing, a neural network model is developed to serve as a global optimization engine. Additionally, a Particle Swarm Optimization (PSO) based local optimizer is employed to improve accuracy. PSO is a simulation-based optimization platform. However, here, it uses the trained NN model instead of a SPICE simulator to get the necessary data.

C. Challenges and Opportunities in Topology Selection

Our vision revolves around a design automation framework that decouples the tasks of topology selection and topology sizing in a data-driven approach. At its core, this framework features a library comprising a multitude of design data points, what we refer to as the pre-sized library of topologies. Based on the given specifications, the goal is to find the most optimal design point from this library. This is a high-dimensional problem as each performance metric within the topology adds a dimension, and a typical analog circuit can have more than a dozen specifications.

1) Topology Dataset Generation: Given a library of topologies, a simple technique to construct a diverse array of design points may involve running a circuit sizing tool with multiple distinct configuration of objectives and constraints on every topology. However, for every new technology, the library must be built from scratch. The simulation cost of such effort, particularly for large and complex circuits with many design parameters and performance metrics, can be high. To tackle this, ML algorithms that are capable of generating synthetic data points from the existing simulation data can be explored. For example, Variational Autoencoders (VAEs) [36], and Generative Adversarial Networks (GANs) [37] have demonstrated potential in generating diverse types of complicated data, including text, images [38], circuit layout strategies [39], time-series [40].

2) Topology Data Visualization: Due to the multi-dimensional nature of the problem, manually selecting an optimal topology from a library of large number of topologies is a challenging task. Nevertheless, if the high-dimensional data can effectively be transformed into low-dimensional space, such as 2/3D, it can serve two valuable purposes.

- Firstly, it can facilitate easier visualization.
- Secondly, it can provide a global perspective, allowing us to understand the relative position of topologies.

Principal component analysis (PCA) is a statistical technique for reducing the dimensionality of a dataset. This is accomplished by linearly transforming the data into a new coordinate system where most of the variation in the data can be described with fewer dimensions than the initial data [41]. The principal component computation involves eigendecomposition of the data covariance matrix, which can be expensive for large dataset [42]. The problem of transforming extensive high-dimensional dataset into low dimension can also be solved using graph embedding algorithms, which is a popular concept in machine learning community [43][42][44]. In recent times, scalable algorithms have been developed to solve the graph embedding problem [44][42].

3) Optimal Topology Selection: While 2/3D-embedding can offer valuable insights into the relative positions of topologies, it alone is not sufficient to identify the optimal topology. We need to explore algorithms that are capable of identifying potential topology candidates or specific region or cluster of topologies within the 2/3D space. Within this probable location or clusters of candidates, nearby topologies can be examined to identify the one that best meets the target specifications.

V. Conclusions

We introduced a new perspective on analog design automation. We first discussed recent advancements in ML-driven analog circuit sizing, layout automation and layout aware sizing techniques. Subsequently, we delved into recent ML-based works that perform topology generation, selection and sizing simultaneously. Following that we outlined our vision for an end-to-end analog design automation framework, at the core of which lies an expansive repository of multitude of design data points of pre-sized topologies. We introduced the problem of optimal topology selection from this high-dimensional dataset and explored machine learning algorithms that could transform it into more accessible visual representations, potentially helping in the selection.

VI. Acknowledgement

This work was supported in part by NSF under grants CCF-1704758 and CCF-2112665, SRC under task 3160.007, and an equipment donation from Nvidia.

References

[1] G. G. E. Gielen and R. A. Rutenbar, "Computer-Aided Design of Analog and Mixed-Signal Integrated Circuits," *Proceedings of the IEEE*, vol. 88, no. 12, pp. 1825–1854, 2000.

[2] A. F. Budak, S. Zhang, M. Liu, W. Shi, K. Zhu, and D. Z. Pan, *Machine learning for analog circuit sizing*. Springer, 2022.

[3] P. Mandal and V. Visvanathan, "CMOS op-amp sizing using a geometric programming formulation," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 20, no. 1, pp. 22–38, 2001.

[4] S. Maji and P. Mandal, "Efficient Approaches to Overcome Non Convexity Issues in Analog Design Automation," in *International Symposium on Quality Electronic Design*, 2012.

[5] S. Zhang, W. Lyu, F. Yang, C. Yan, D. Zhou, and X. Zeng, "Bayesian Optimization Approach for Analog Circuit Synthesis Using Neural Network," in *Design, Automation and Test in Europe Conference*, 2019.

[6] A. F. Budak, M. Gandara, W. Shi, D. Z. Pan, N. Sun, and B. Liu, "An Efficient Analog Circuit Sizing Method Based on Machine Learning Assisted Global Optimization," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 5, pp. 1209–1221, 2021.

[7] A. F. Budak, K. Zhu, and D. Z. Pan, "Practical Layout-Aware Analog/Mixed-Signal Design Automation with Bayesian Neural Networks," in *International Conference on Computer-Aided Design*, 2023.

[8] K. Settaluri, A. Haj-Ali, Q. Huang, K. Hakhamaneshi, and B. Nikolic, "AutoCkt: Deep Reinforcement Learning of Analog Circuit Designs," in *Design, Automation and Test in Europe*, 2020.

[9] K. Settaluri, Z. Liu, R. Khurana, A. Mirhaj, R. Jain, and B. Nikolic, "Automated Design of Analog Circuits Using Reinforcement Learning," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 41, no. 9, pp. 2794–2807, 2022.

[10] A. F. Budak, P. Bhansali, B. Liu, N. Sun, D. Z. Pan, and V. K. Chandramouli, "DNN-Opt: An RL Inspired Optimization for Analog Circuit Sizing using Deep Neural Networks," in *Design Automation Conference*, 2021.

[11] A. F. Budak, D. Smart, B. Swahn, and D. Z. Pan, "APOSTLE: Asynchronously Parallel Optimization for Sizing Analog Transistors Using DNN Learning," in *Asia and South Pacific Design Automation Conference*, 2023.

[12] H. Wang, K. Wang, J. Yang, L. Shen, N. Sun, H. Lee, and S. Han, "GCN-RL circuit designer: transferable transistor sizing with graph neural networks and reinforcement learning," in *Design Automation Conference*, 2020.

[13] H. Chen, M. Liu, X. Tang, K. Zhu, A. Mukherjee, N. Sun, and D. Z. Pan, "MAGICAL 1.0: An Open-Source Fully-Automated AMS Layout Synthesis Framework Verified With a 40-nm 1GS/s $\Delta\Sigma$ ADC," in *Custom Integrated Circuits Conference*, 2021.

[14] K. Zhu, H. Chen, M. Liu, and D. Z. Pan, "Tutorial and Perspectives on MAGICAL: A Silicon-Proven Open-Source Analog IC Layout System," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 70, pp. 715–720, 2023.

[15] K. Kunal, M. Madhusudan, A. K. Sharma, W. Xu, S. M. Burns, R. Harjani, J. Hu, D. A. Kirkpatrick, and S. S. Sapatnekar, "ALIGN: Open-Source Analog Layout Automation from the Ground Up," in *Design Automation Conference*, 2019.

[16] T. Dhar, K. Kunal, Y. Li, M. Madhusudan, J. Poojary, A. K. Sharma, W. Xu, S. M. Burns, R. Harjani, J. Hu, D. A. Kirkpatrick, P. Mukherjee, S. Yaldiz, and S. S. Sapatnekar, "ALIGN: A System for Automating Analog Layout," *IEEE Design Test*, vol. 38, no. 2, pp. 8–18, 2021.

[17] K. Hakhamaneshi, N. Werblun, P. Abbeel, and V. Stojanovic, "BagNet: Berkeley Analog Generator with Layout Optimizer Boosted with Deep Neural Networks," in *International Conference on Computer-Aided Design*, 2019.

[18] H. Ren, G. Kokai, W. Turner, and T. Ku, "ParaGraph: layout parasitics and device parameter prediction using graph neural networks," in *Design Automation Conference*, 2020.

[19] B. Shook, P. Bhansali, C. Kashyap, C. Amin, and S. Joshi, "ML-Parest: machine leaning based parasitic estimation for custom circuit design," in *Design Automation Conference*, 2020.

[20] M. Liu, W. J. Turner, G. F. Kokai, B. Khailany, D. Z. Pan, and H. Ren, "Parasitic-Aware Analog Circuit Sizing with Graph Neural Networks and Bayesian Optimization," in *Design, Automation and Test in Europe Conference*, 2021.

[21] T. McConnaghy, P. Palmers, M. Steyaert, and G. Gielen, "Trustworthy Genetic Programming-Based Synthesis of Analog Circuit Topologies Using Hierarchical Domain-Specific Building Blocks," *IEEE Transactions on Evolutionary Computation*, vol. 15, no. 4, pp. 557–570, 2011.

[22] Z. Zhao and L. Zhang, "An Automated Topology Synthesis Framework for Analog Integrated Circuits," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 12, pp. 4325 – 4337, 2020.

[23] Z. Dong, W. Cao, M. Zhang, D. Tao, Y. Chen, and X. Zhang, "CktGNN: Circuit Graph Neural Network for Electronic Design Automation," in *International Conference on Learning Representations*, 2023.

[24] J. Lu, Y. Li, F. Yang, L. Shang, and X. Zeng, "High-Level Topology Synthesis Method for Δ-Σ Modulators via Bi-level Bayesian Optimization," *IEEE Transactions on Circuits and Systems II: Express Briefs*, 2023.

[25] S. Fan, N. Cao, S. Zhang, J. Li, X. Guo, and X. Zhang, "From specification to topology: Automatic power converter design via reinforcement learning," in *International Conference On Computer-Aided Design*, 2021.

[26] P. Veselinovic, D. Leenaerts, W. Van Bokhoven, F. Leyn, F. Proesmans, G. Gielen, and W. Sansen, "A Flexible Topology Selection Program as Part of an Analog Synthesis System," in *European Design Automation and Test Conference*, 1995.

[27] T. McConaghy, P. Palmers, G. Gielen, and M. Steyaert, "Automated Extraction of Expert Knowledge in Analog Topology Selection and Sizing," in *International Conference on Computer-Aided Design*, 2008.

[28] S. Maji and P. Mandal, "A Geometric Programming Aided Knowledge based Approach for Analog Circuit Synthesis and Sizing," in *Great Lakes Symposium on VLSI*, 2011.

[29] ——, "A CAD methodology for automatic topology selection & sizing," in *International SOC Conference*, 2011.

[30] M. Fayazi, M. T. Taba, E. Afshari, and R. Dreslinski, "AnGeL: Fully-Automated Analog Circuit Generator Using a Neural Network Assisted Semi-Supervised Learning Approach," *IEEE Transactions on Circuits and Systems—I*, vol. 70, no. 11, pp. 4516–4529, 2023.

[31] S. Maji, S. Dam, and P. Mandal, "Automatic Generation of Saturation Constraints and Performance Expressions for Geometric Programming based Analog Circuit Sizing," in *International Symposium on Quality Electronic Design*, 2011.

[32] Z. Lu, H. Pu, F. Wang, Z. Hu, and L. Wang, "The Expressive Power of Neural Networks: A View from the Width," in *Neural Information Processing Systems*, 2017.

[33] S. M. Burns, H. Chen, T. Dhar, R. Harjani, J. Hu, N. Karmokar, K. Kunal, Y. Li, Y. Lin, M. Liu, M. Madhusudan, P. Mukherjee, D. Z. Pan, J. Poojary, S. Ramprasath, S. S. Sapatnekar, A. K. Sharma, W. Xu, S. Yaldiz, and K. Zhu, *Machine Learning for Analog Layout*. Springer, 2022.

[34] E. Chang, J. Han, W. Bae, W., N. Z., B. N., NikoliC, and E. Alon, "BAG2: a processportable framework for generator-based AMS circuit design," in *Custom Integrated Circuits Conference*, 2018.

[35] E. Goan and C. Fookes, *Bayesian neural networks: An introduction and survey*. Springer, 2020.

[36] D. P. Kingmaand and M. Welling, "Auto-Encoding Variational Bayes," in *International Conference on Learning Representations*, 2014.

[37] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative adversarial nets," in *International Conference on Neural Information Processing*, 2014.

[38] C. Doersch, "Tutorial on variational autoencoders," *arXiv preprint arXiv:1606.05908*, 2016.

[39] K. Zhu, M. Liu, Y. Lin, B. Xu, S. Li, X. Tang, N. Sun, and D. Z. Pan, "GeniusRoute: A New Analog Routing Paradigm Using Generative Neural Network Guidance," in *International Conference on Computer-Aided Design*, 2019.

[40] J. Yoon, D. Jarrett, and M. v. d. Schaar, "Time-series Generative Adversarial Networks," in *Neural Information Processing Systems*, 2019.

[41] I. T. Jolliffe and J. Cadima, "Principal component analysis: a review and recent developments," *Philosophical Transactions of the Royal Society A*, vol. 374, no. 2065, 2016.

[42] A. Grover and J. Leskovec, "node2vec: Scalable Feature Learning for Networks," in *The International Conference on Knowledge Discovery and Data Mining*, 2016.

[43] M. Belkin and P. Niyogi, "Laplacian eigenmaps and spectral techniques for embedding and clustering," in *Neural Information Processing Systems*, 2001.

[44] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, and Q. Mei, "LINE: Large-scale Information Network Embedding," in *International Conference on World Wide Web*, 2015.